

Conceptual Review of Project Elements

Overview

This course has equipped you with the knowledge and experience to create visualizations which exceed the quality created for data-science foundations course. The reasonable expectation is that the quality of project-related visualizations used in the team project reflect integration of these experiences. Moreover, the course has consistently supported conceptual issues related to project management and reproducibility through weekly exercises.

This homework requires you to **refine a plot** and integrate all of the **project workflow and management elements** you have worked on repeatedly over the semester. In essence, the homework represents the final step to ensure you bring together the all of the project workflow elements covered consistently in the course using practical in-class exercises over the course of the semester (e.g., building and documentation script, sourcing scripts from other files, building web pages, and managing files with Git). These workflow elements covered throughout the semester are reviewed below.

Essential Project File Types

For various exercises and homework assignments, you have been working with `.R` scripts, `.Rds` files, `.qmd` files, and `.yaml` files for building your team's webpage.

`.R` Script Files

A. Data-Related Script Files Data manipulation and associated scripts were introduced as early as Exercise 2 and HW01. As an early step, the code lead/s worked on assembling the data for the rest of the team. Data scripts will load relevant libraries for working with the data. The team should have:

- a **data-reading script** for accessing/reading data using an API or reading and combining multiple files
- a **data-cleaning script** for wrangling the data, fixing problematic variables, creating new variables, and organizing the data in a clean data frame to prevent redundancies in separate plotting scripts; depending on the
- other scripts that may involve some data aggregation if elements of the project

B. Function-Related Script Files All team members worked on a plot-saving script that would converted `ggplot` objects to `.png` files. Only one version needed.

- a function script for saving plots as `.png` files (see HW03)
- an optional theme function for ensuring all fonts and axes are uniform (recommended; related to Figure Design/Theme Topic)

C. Plot-Related Script Files All team members have used the `starter_script.R` to create a plotting script for building all visualizations for the website. Plotting scripts will load libraries, the plot-saving script, and load `.Rds` data files created by the data-related scripts in order to ensure the plot will be created with the most up-to-date data.

D. Page Files

All team members have created `.qmd` files for building the web pages and for calling the `.png`. For some exercises/homework, you use the `source()` function in your `.qmd` files so that you could efficiently call your plotting scripts.

As a sequential workflow, page files that contain embedding visualization will require code blocks to:

- source the data-related `.R` scripts for preparing the data needed for the plotting script;
- source the plot-related `.R` script(s) that call and use the `.Rds` data files created by the data-related scripts needed to produce the `.png` files for the website;
- call the `.png` created by the plotting-script file from `figs/` and embed in the page (the last step); see examples below

Code Block For Calling a Plot (No Caption):

```
```${r}
#| message: false # to hide messages
#| warning: false # to hide warnings
#| include: false # to prevent code from showing (show output only)
#| fig.align: "center" # to center plot on page if desired
#| out.width: "10%" # or some other % appropriate for your site

knitr::include_graphics(here::here("figs", "beavis_stag_ht_championship.png"))
```
```

Code Block For Calling a Plot (Add a Caption):

```
```{r}
plot with a caption
#| message: false # to hide messages
#| warning: false # to hide warnings
#| include: false # to prevent code from showing (show output only)

#| fig.cap: "Your custom message here"
#| fig.align: "center" # to center plot on page if desired
#| out.width: "100%" # or some other % appropriate for you site

knitr::include_graphics(here::here("figs", "beavis_stag_ht_chamionship.png"))
```
```

E. A .yaml Website Instruction File

A YAML file that defines the design structure, theme, and color scheme for creating the HTML webpage files.

Review of Course Exercises and Concept Coverage

The in-class exercises familiarized you with project workflow elements and provided opportunities for you to reinforce those elements. In order to support your learning incrementally and motivate integration of these elements into your weekly development of the course project components, these workflow elements were built into several exercises. Those workflow elements are summarized below.

- **Exercise 1:** On *Team Project and Git Setup* involved using Git and configuring your personal and dev branches of the repo, two branches that were integral to your weekly code development and team file communication.
- **Exercise 2:** On *Understanding Grouping and Summarizing Data Frames* further involved building your `starter_script.R` for data cleaning and writing data as `.Rds` files rather than `.csv` or `.xlsx` data files due to their compressed size and portability with saving variable type information. The exercise also supported project organization by supporting your understanding of saving data files in specific directories. Staging, committing, and pushing files to your personal branch was also initialized.
- **Exercise 3:** On *Grouping, Summarizing, and Plotting (with Git)* focused on intentional file creating and provided more practice modifying `starter_script.R` and documenting sections for loading libraries, reading data, and writing data (Part 1). You also built a script to load libraries, read cleaned `.Rds` files using the `{here}` library to ensure file paths are not hard coded, and plot data (Parts 2 and 3). Finally, you further worked with Git (Part 4).

- **Exercise 4:** On *Visualizing Associations and Mapping Aesthetics* provided additional practice using the `starter_script_file.R` to build well documented and easy to follow simple scripts as well as working with Git branches. By this time, you created a habit for creating, naming, and saving your `.R` scripts that served as the foundation for the project code.
- **Exercise 5:** *Spatial Position and Adjustment* provided more experience working with the `starter_script.R` to document and section code as well as splitting off separate plots into separate scripts (rather than coding busy scripts with many working elements) named in a way to ensure the script reflects the intentions of the script and the plot. This exercise gave you practice compartmentalizing code and plot scripts to reduce and facilitate troubleshooting issues.
- **Exercise 6:** *Color Scales and Palettes* provided steps for controlling colors used in visualizations, ensuring reproducibility of color palettes across plots, and using libraries that simplify this process. Colors were considered for the team website for the palette to be used in plots and website themes. At this phase, the team was thinking about color design elements and building plots in ways that would allow for color reproducibility across plots.
- **Exercise 7:** *Coordinates, Axes, and Scales* provided practice labeling scales more efficiently and appropriately for tackling the data-to-ink ratio as well as additional practice using data-frame related color assignment based on variable identity.
- **Exercise 8:** *Mastering `stat_summary()`* provided examples for creating data summaries with fine-grained, or very dis-aggregated data, providing contrast with using data `{dplyr}`-based data data frame summaries. You practiced using the `stat_summary()` along with built-in as well as with custom functions for various use cases and for handling some cases of uncertainty.
- **Exercise 9:** *More/Multiple Distributions* provided extensions of density plots and examples for adding additional plots to axis margins, something perhaps more relevant diagnosing data characteristics for modeling. The exercise challenged you to extract information from the visualization that were not depicted visually in an effort to reinforce perceptual challenges and encourage thinking about the user.
- **Exercise 10:** *Uncertainty and Variability* emphasized the use of `stat_summary()` for communicating information beyond point estimates, which on their own offer a myopic view of data (the tree in the forest) and overlook distributional information relevant for correctly interpreting data and for making statistical comparison across groups. The exercise also challenged you to think about plot layering.
- **Exercise 11:** *Visualizing Trends* extended the experience of examining relationships in data with the focus on linear and LOESS trend types.
- **Exercise 12:** *Small Multiples* offered experience reducing the clutter associated with busy or complex visualizations by creating plot replicates across levels of a grouping

variable. Separating data from a single plot posed additional challenges with unaligned axes to be considered when deciding on plot types.

Across the exercises, you consistently practiced implementing these workflow elements weekly which allowed you to fine-tune the quality of your coding and code organization incrementally, thereby avoiding an end-of-semester code overhaul. Throughout the course and as early as HW01 and in do's and don'ts in the project website files, you also repeatedly practiced working with an organized directory structure and organizing files into designate directories (e.g., `data/`, `src/data/`, `src/figs/`, `figs/`, `pages/`, etc.) and using `{here}` for managing file paths and avoiding hard coding of any paths.

Workflow Integration

Taken together, the regular weekly exercises have offered you extensive practice using a variety of R libraries. Beyond the computational experience, exercises have supported your implementation of relevant and necessary conceptual elements of data-science projects including:

- (a) maintaining an *organized project directory structure*, with
- (b) files created to carry out specific use cases and saved in appropriate directories,
- (c) *naming data, script, and page files meaningfully* based on their intended goals so that their names communicate their intended action and their names are machine readable in general and specifically for case-sensitive languages like R,
- (d) *authoring and and script files* to provide clarity about their intended goals and steps with relevant sections flagged clearly for troubleshooting and reproducibility,
- (e) *sourcing script files* (the workhorses for data-science projects) within other script files, if necessary, and within website `.qmd` files in order to carry out tailored execution of code in a way that facilitates, execution flexibility (e.g., easy to comment out), code troubleshooting (diagnosing and fixing problems), reduces repo/Git conflicts, and supports reproducibility across collaborator systems.

This entire workflow supported in the exercises allows you to ensure a project will render from start to finish without error. For example, upon each rendering, your project will:

- (a) read/update data or integrate new data (e.g., new data files, new annual, quarterly, monthly, or daily data added to the project),
- (b) combine/clean/wrangle old and new data to accommodate changes,
- (c) generate updated versions of plots using new data, and
- (d) build the pages for the website by pushing them to the repository for publishing on the Internet. Each team has seen their website live on the Internet. The final step will be to leverage your branch merging experience which was set forth as part of your weekly workflow habit (see HW01). Eventually, the team will need to

- (e) merge all relevant files to the **main** branch and ensure the branch contains only project-relevant files saved with appropriate names and in appropriate directories so that the website will be built on the main branch and made available for GitHub Pages. Currently, GitHub Pages is serving up page files on the design lead's personal branch but this change will soon be implemented.

Note: In order to ensure reproducibility, I encourage the team to build the website with a subset of data (e.g., a missing year, month, quarter) and then rebuild the website with the data added back to ensure files are updating accordingly.